PRODUCT REQUIREMENTS DOCUMENT

DelegateTracker — Smart Contract Delegatecall Security Analysis

Scope: Group 1 — Attack Path Search | Group 2 — Attack Path Validation + Product

Document field	Details
Document type	Product Requirements Document (PRD)
Product	DelegateTracker (working title)
Version	1.0 — Initial implementation specification
Primary users	Smart-contract developers, security researchers, auditors, students
Core workflow	Solidity source → candidate attack paths → evidence-based validation → security report

Purpose: define the product scope, requirements, interfaces, architecture, acceptance criteria, and implementation roadmap for both groups.

1. Executive Summary

DelegateTracker is a smart-contract security analysis product focused on investigating potentially unsafe use of Solidity's **delegatecall**. The system is divided into two cooperating groups. Group 1 performs static analysis to discover suspicious function/data-flow paths and emits a standardized **CandidateAttackPath[]**. Group 2 consumes those candidates, executes targeted symbolic/dynamic validation, records runtime evidence, evaluates path constraints and storage interactions, and produces **ValidationResult[]** alongside a visual attack path and security report.

The system must distinguish a suspicious candidate from a confirmed exploit. Presence of delegatecall, matching a storage slot, or reaching a sensitive operation alone must not automatically be treated as proof; confirmation must use a documented, vulnerability-specific set of conditions and execution evidence.

2. Product Vision and Goals

2.1 Vision

Provide an explainable workflow that connects static attack-path discovery with evidence-based validation, helping users understand whether a suspected delegatecall path is feasible and why it is considered risky.

2.2 Goals

- Accept Solidity contracts and analyze them for candidate delegatecall-related attack paths.
- Represent candidates in a versioned, machine-readable schema shared by both groups.
- Validate candidates using an EVM symbolic-execution engine and targeted execution strategy.
- Capture execution evidence, including storage reads/writes, call context, path constraints, and sensitive-operation reachability where supported.
- Return explicit outcomes: confirmed, not confirmed, inconclusive, or execution error.
- Visualize the path and provide reproducible technical reports.
- Use AI only to explain validated evidence; AI must not be the source of the security verdict.

2.3 Non-goals for the first release

- Guaranteeing detection of every Solidity/EVM vulnerability.
- Automatically exploiting or interacting with live mainnet contracts.
- Replacing a professional audit, formal verification, or developer review.
- Supporting every compiler version, proxy architecture, and Solidity storage construct in the MVP.
- Allowing an LLM to independently declare a contract safe or vulnerable.

3. Users and Primary Use Cases

User	Need / use case
Smart-contract developer	Upload a contract, inspect suspicious paths, understand evidence, and receive remediation guidance.
Security researcher / auditor	Review candidate paths, inspect runtime traces and constraints, and export findings.
Student / project evaluator	Demonstrate the separation of static search and validation with repeatable test contracts.
Integration client / Group 1	Send CandidateAttackPath[] and receive stable ValidationResult[] objects.

4. System Boundary and End-to-End Workflow

```
Solidity source + compiler settings
|
v
GROUP 1: STATIC ATTACK PATH SEARCH
compiler/parser -> AST -> functions
-> CFG/call graph -> state READ/WRITE analysis
-> delegatecall + sensitive-operation detection
-> pattern matching -> candidate path generation
|
v
CandidateAttackPath[]
|
v
GROUP 2: PATH VALIDATION + PRODUCT
schema validation -> artifact loading
-> symbolic execution / targeted path exploration
-> runtime trace + storage READ/WRITE evidence
-> constraint evaluation -> vulnerability-specific rules
-> ValidationResult[]
|
+--> path visualization
+--> security report
+--> evidence-grounded AI explanation
```

Boundary rule: Group 1 owns candidate discovery and static metadata. Group 2 owns execution-based validation and user-facing outputs. Shared schema definitions and test fixtures should be maintained collaboratively.

5. Functional Requirements — Group 1: Attack Path Search

G1-FR-01 — Solidity Input and Compilation

The system shall accept Solidity source files and compiler settings, validate input type/size, identify the compiler version where possible, and compile or parse the source. Compilation failures shall be returned with actionable diagnostics.

G1-FR-02 — AST and Function Extraction

The system shall extract contracts, functions, modifiers, parameters, visibility, relevant statements, state variables, and source locations from compiler/parser output.

G1-FR-03 — CFG and Call Graph

The system shall build a control-flow graph for function-level execution structure and a call graph connecting internal/external calls where statically resolvable. Unknown or dynamic edges shall be marked as unresolved rather than silently omitted.

G1-FR-04 — State READ/WRITE Analysis

The system shall identify state-variable reads and writes per function, including source locations and confidence. The implementation should distinguish direct variable access from indirect or dynamically indexed storage where feasible.

G1-FR-05 — Delegatecall Detection

The system shall identify delegatecall sites, target expressions, call context, and relevant source locations. Detection of delegatecall alone shall not constitute a vulnerability finding.

G1-FR-06 — Sensitive-Operation Detection

The system shall detect configured sensitive operations such as value-transfer calls, low-level calls, selfdestruct where applicable to the compiler/EVM version, and security-relevant conditional/judgment operations.

G1-FR-07 — Vulnerability Pattern Matching

The MVP shall implement three documented candidate patterns: currency-sending anomaly, self-destruct anomaly, and judgment-condition anomaly. Each pattern shall define required evidence, exclusions, and known limitations.

G1-FR-08 — Candidate Attack-Path Generation

The system shall connect function calls, state-variable data flow, delegatecall context, and sensitive operations to produce bounded candidate paths. Each candidate shall include provenance, source locations, pattern identifier, and confidence/limitations.

G1-FR-09 — Basic Storage Layout

The system shall attach expected storage-slot information where compiler storage-layout output or reliable static derivation is available. Unknown or complex slots shall be represented as unknown/partial, not guessed.

G1-FR-10 — Group 1 Output

The system shall emit a versioned CandidateAttackPath[] JSON payload and diagnostics describing omitted, unresolved, or low-confidence paths.

6. Functional Requirements — Group 2: Validation + Product

G2-FR-01 — Candidate Ingestion and Normalization

The system shall accept CandidateAttackPath[] through an API or JSON upload, validate schema version and required fields, normalize accepted records, and reject malformed payloads with field-level errors.

G2-FR-02 — Contract Artifact Resolution

For each candidate, the system shall obtain the matching source, compiler settings, ABI, bytecode, and deployment/setup information required by the execution backend. Artifacts must be tied to a source/compiler hash to avoid validating the wrong build.

G2-FR-03 — Symbolic Execution

The system shall execute targeted candidate paths using an EVM symbolic-execution engine such as Manticore, subject to supported versions and resource limits. The execution adapter shall isolate engine-specific APIs from the validation rules.

G2-FR-04 — Candidate Path Tracking

The system shall record whether expected functions/operations were reached, whether execution reverted, and whether path exploration was incomplete. Function names alone are not always unique; source locations, selectors, program counters, and call context should be used when available.

G2-FR-05 — Runtime READ/WRITE Capture

The system shall capture SLOAD/SSTORE or equivalent trace events where the backend exposes them. Events should include sequence number, storage-owning address, executing code address/context, slot, value when available, program counter, call depth, transaction/state identifier, and symbolic/concrete status.

G2-FR-06 — Storage-Slot Matching

The system shall compare Group 1's expected slot information with runtime storage events and return matched, mismatched, unknown, or partial. Mapping/array/struct slots must account for Solidity's storage-layout rules; unsupported cases shall be marked inconclusive.

G2-FR-07 — Constraint Validation

The system shall retain or summarize path constraints and determine satisfiability through the execution engine's solver interface. It shall preserve solver status and any generated witness values, subject to safe report redaction.

G2-FR-08 — Vulnerability Confirmation

The system shall apply documented, pattern-specific confirmation rules using execution feasibility, data-flow evidence, storage context, and sensitive-operation reachability. It shall not use a single heuristic as universal proof.

G2-FR-09 — Classification and Status

The system shall classify findings into supported pattern categories and assign evidence-based severity. Results shall distinguish CONFIRMED, NOT_CONFIRMED, INCONCLUSIVE, and ERROR. 'Not confirmed' shall not be displayed as a guarantee that the contract is safe.

G2-FR-10 — Visualization

The system shall display the candidate path as a graph, including functions, state variables/storage slots, READ/WRITE edges, delegatecall boundary, sensitive operation, and evidence status. Users shall be able to inspect event details and source locations where available.

G2-FR-11 — Security Report

The system shall generate a report containing contract/build identity, candidate ID, pattern, path, verdict, evidence, constraints/solver status, limitations, severity rationale, and remediation notes. JSON is required; HTML/PDF is a product target.

G2-FR-12 — AI Explanation

The system may generate a natural-language explanation from the structured validation result. The prompt shall treat source and contract text as untrusted data, and the explanation must not change the deterministic verdict. Unknown evidence must be stated as unknown.

G2-FR-13 — Job and Result Management

The API shall return a job/result identifier, expose status for long-running validation, and support retrieval of results and reports. Cancellation, timeouts, and failed jobs shall be represented explicitly.

7. Shared Data Contracts

7.1 CandidateAttackPath — illustrative schema

```
{
"schema_version": "1.0",
"id": "CAP-001",
"contract": {
"name": "Wallet",
"source_hash": "sha256:...",
"compiler_version": "0.8.x"
},
"delegatecall": {
"detected": true,
"function": "execute",
"source_location": {"start": 100, "length": 24}
},
"path": {
"functions": ["execute", "withdraw"],
"edges": []
},
"data_flow": {
"variable": "owner",
"write_function": "execute",
"read_function": "withdraw"
},
"sensitive_operation": {
"type": "value_transfer",
"function": "withdraw"
},
"pattern": "currency_sending_anomaly",
"storage": {
"expected_slot": "0",
"slot_status": "known"
},
"limitations": []
}
```

This is an illustrative contract, not a mandatory final schema. The groups must agree on exact field names, types, nullability, source-location convention, storage-slot encoding, and schema version before parallel implementation.

7.2 ValidationResult — illustrative schema

```
{
"schema_version": "1.0",
"id": "VR-001",
"candidate_id": "CAP-001",
"status": "CONFIRMED",
"verified": true,
"pattern": "currency_sending_anomaly",
"path_evidence": {
"expected_steps_reached": [],
"sensitive_operation_reached": true
},
"storage_validation": {
"expected_slot": "0",
"runtime_reads": [],
"runtime_writes": [],
"match_status": "MATCHED"
},
"constraints": {
"status": "SAT",
"witness_available": false
},
"severity": "HIGH",
"limitations": [],
"report_id": "REPORT-001"
}
```

Do not encode EVM storage slots as ordinary JSON numbers in all cases: use a canonical string/hex representation to avoid ambiguity and preserve large values. Do not include secrets or unrestricted private keys in witness output.

8. Non-Functional Requirements

Category	Requirement
Correctness	Every verdict must be traceable to evidence and documented rules; uncertain cases must be INCONCLUSIVE.
Security	Run untrusted contracts in isolated worker processes/containers with CPU, memory, disk, and time limits; no live-chain transactions by default.
Reliability	A single malformed candidate or failed execution must not crash the entire batch.
Performance	Set configurable per-candidate and per-batch limits; record timeouts and explored-state counts where available.
Reproducibility	Record source hash, compiler version/settings, engine version, configuration, and test seed/environment.
Usability	Show plain-language status, detailed technical evidence, and limitations separately.
Maintainability	Keep the EVM engine behind an adapter; keep validation rules separate from API/UI code.
Privacy	Avoid transmitting contract source to external AI services without explicit configuration/consent; redact sensitive data from logs.
Compatibility	Publish a supported Solidity/compiler/EVM matrix and report unsupported builds clearly.

9. Suggested Technical Architecture

```
Frontend (React + graph component)
|
v
Backend API (FastAPI)
|-- upload/schema validation
|-- job/status/result endpoints
|-- report and explanation endpoints
|
v
Validation Orchestrator
|-- candidate normalizer
|-- artifact resolver
|-- EVM engine adapter (e.g., Manticore)
|-- runtime trace normalizer
|-- storage matcher
|-- constraint evaluator
|-- pattern-specific confirmation rules
|-- classifier/result builder
|
+--> JSON/HTML/PDF reports
+--> evidence-grounded AI explanation

Group 1 pipeline:
Solidity -> compiler/parser -> AST -> CFG/call graph
-> READ/WRITE and delegatecall analysis -> patterns
-> CandidateAttackPath[]
```

Suggested MVP stack: Solidity compiler/solc for artifacts; Python for analysis orchestration; Manticore or a verified compatible EVM symbolic-execution backend; FastAPI for the API; React for the UI; React Flow or Graphviz for graphs; JSON first, then HTML/PDF reports. Verify tool compatibility with the selected Python and Solidity versions before locking dependencies.

10. API Requirements

Method / endpoint	Purpose
POST /api/candidates/validate	Submit CandidateAttackPath[] and create a validation job.
GET /api/jobs/{job_id}	Get queued/running/completed/failed status.
GET /api/results/{result_id}	Retrieve a ValidationResult.
GET /api/results/{result_id}/path	Retrieve graph-ready path nodes and edges.
GET /api/results/{result_id}/report	Retrieve or generate the security report.
POST /api/results/{result_id}/explanation	Generate an evidence-grounded AI explanation.

If Group 1 and Group 2 run in separate services, use the same versioned JSON schema and include request IDs, candidate IDs, build hashes, and structured errors. Authentication and upload-size limits should be added before exposing the API beyond localhost.

11. Acceptance Criteria

11.1 Group 1 acceptance

- Given a supported Solidity test contract, the pipeline produces parse/compile diagnostics and extracts relevant functions.
- Delegatecall sites and configured sensitive operations include source locations where available.
- At least the three agreed patterns can emit candidate paths with required provenance.
- The output validates against the shared CandidateAttackPath schema.
- Unsupported/dynamic cases are marked unresolved instead of silently treated as safe.

11.2 Group 2 acceptance

- A valid candidate JSON payload is accepted; malformed payloads receive structured validation errors.
- A known reachable test path yields runtime evidence and a reproducible result.
- READ/WRITE events preserve storage context and call depth where supported by the backend.
- Storage comparison can report MATCHED, MISMATCHED, PARTIAL, or UNKNOWN.
- Constraint outcomes distinguish SAT, UNSAT, UNKNOWN, and solver error/timeout.
- The rule engine returns CONFIRMED only when the relevant pattern's required evidence is satisfied.
- Timeouts, unsupported constructs, and incomplete exploration produce INCONCLUSIVE or ERROR rather than false certainty.
- A user can inspect the path graph and export a report containing evidence and limitations.
- AI explanations accurately reflect the structured result and cannot override it.

11.3 End-to-end acceptance

- A Group 1 candidate can be passed to Group 2 without manual field editing.
- The same source/build hash is used during validation as during candidate generation.
- Safe and vulnerable fixtures are both tested; outcomes are recorded in a regression suite.
- The final demo shows one confirmed case, one not-confirmed case, and one inconclusive/unsupported case.

12. Testing and Evaluation Plan

Test layer	Coverage
Unit	Schema validation, normalization, storage matching, rule evaluation, classification, report serialization.
Engine integration	Contract deployment/setup, symbolic inputs, path exploration, trace collection, solver statuses.
End-to-end	Group 1 JSON → Group 2 execution → ValidationResult → graph/report.
Regression corpus	Small controlled vulnerable and safe Solidity contracts for each supported pattern.
Robustness	Malformed JSON, missing artifacts, compiler mismatch, timeout, solver unknown, unsupported storage patterns.
Evaluation metrics	Precision, recall where ground truth exists, false-positive/false-negative counts, runtime, timeout rate, candidate coverage.

Build a labeled test corpus and document the ground truth for each fixture. A successful symbolic execution is not automatically proof of exploitability: test whether attacker-controlled conditions, authorization assumptions, deployment state, and the actual sensitive effect are represented.

13. Risks and Mitigations

Risk	Mitigation
Symbolic path explosion	Validate only candidate paths, bound depth/state/time, prioritize candidate-specific branches.
Incorrect storage mapping	Use compiler storage layout where possible; mark dynamic/mapping cases partial or unknown until supported.
False confirmation from slot match alone	Use pattern-specific multi-condition rules and record complete evidence.
Engine/compiler incompatibility	Maintain a tested version matrix and fail clearly on unsupported builds.
Group interface drift	Version the schema, maintain shared fixtures, and run contract tests in CI.
AI hallucination or prompt injection	Generate explanations only from normalized evidence; treat source code as untrusted; preserve deterministic verdict.
Malicious uploaded contracts	Isolate execution, disable network access by default, and enforce resource limits.

14. Implementation Roadmap

Milestone	Deliverables	Exit condition
M0 — Contract	Shared schemas, pattern definitions, sample contracts, tool/version choices.	Both groups agree on fields and test fixtures.
M1 — Group 1 MVP	Parse/compile, function extraction, delegatecall and sensitive-operation detection, initial candidate output.	Schema-valid candidates from controlled fixtures.
M2 — Group 2 engine spike	Deploy/load fixture, execute one candidate with chosen engine, collect basic trace.	Reproducible execution from CLI.
M3 — Validation core	READ/WRITE capture, storage matcher, constraints, pattern rules, ValidationResult.	Confirmed/not-confirmed/inconclusive cases pass tests.
M4 — Integration	End-to-end pipeline and regression tests.	No manual transformation between groups.
M5 — Product UI	Upload, job status, path graph, evidence panel, JSON report.	Evaluator can inspect one full result.
M6 — Final features	PDF/HTML report, AI explanation, evaluation metrics, documentation.	Demo, limitations, and test results documented.

15. Team Ownership

Owner	Responsibilities
Group 1	Static pipeline, candidate generation, source locations, expected storage metadata, schema-compliant output.
Group 2	Candidate ingestion, execution backend, runtime evidence, storage/constraint validation, verdicts, classification, reports, visualization, AI explanation.
Joint	Schema/versioning, pattern semantics, test corpus, build identity, end-to-end tests, acceptance review.

16. Definition of Done

- Both groups implement the shared, versioned schema and publish example payloads.
- Every verdict includes evidence, engine status, and limitations.
- Confirmed/not-confirmed/inconclusive outcomes are distinct and tested.
- Execution of untrusted contracts is isolated and resource-limited.
- A regression suite includes both vulnerable and safe examples.
- A reproducible demo and setup instructions are included in the README.
- The report and AI explanation are generated from the stored ValidationResult, not independently inferred from raw source.

17. Glossary

Term	Meaning
AST	Abstract Syntax Tree: structured representation of source code.
CFG	Control-Flow Graph: possible execution flow through a function.
Call graph	Graph of possible calls between functions.
delegatecall	EVM call type that executes target code in the caller's storage/context.
CandidateAttackPath	A suspicious path emitted by Group 1 for further validation.
Symbolic execution	Execution using symbolic inputs and path constraints rather than only concrete values.
SLOAD / SSTORE	EVM instructions for reading from and writing to storage.
SAT / UNSAT	Solver results indicating whether constraints are satisfiable or unsatisfiable.
ValidationResult	Structured outcome and evidence produced by Group 2.

Appendix A — Sample Result Status Semantics

Status	Meaning
CONFIRMED	Pattern-specific required evidence was observed and constraints are feasible under the modeled assumptions.
NOT_CONFIRMED	The tested candidate did not meet the confirmation conditions within the completed analysis scope; this does not prove overall safety.
INCONCLUSIVE	Analysis could not determine the outcome because of timeout, unsupported features, incomplete path exploration, or unknown solver/storage result.
ERROR	The validation job failed due to invalid artifacts, engine failure, or an internal error.

Appendix B — MVP Scope Recommendation

For a realistic first release, support a documented subset of Solidity/compiler versions and a small number of testable patterns. Start with schema validation, one contract setup, one candidate path, trace capture, and a deterministic ValidationResult. Add broader storage constructs, graph UI, PDF reporting, and AI explanations only after the validation core is repeatable.